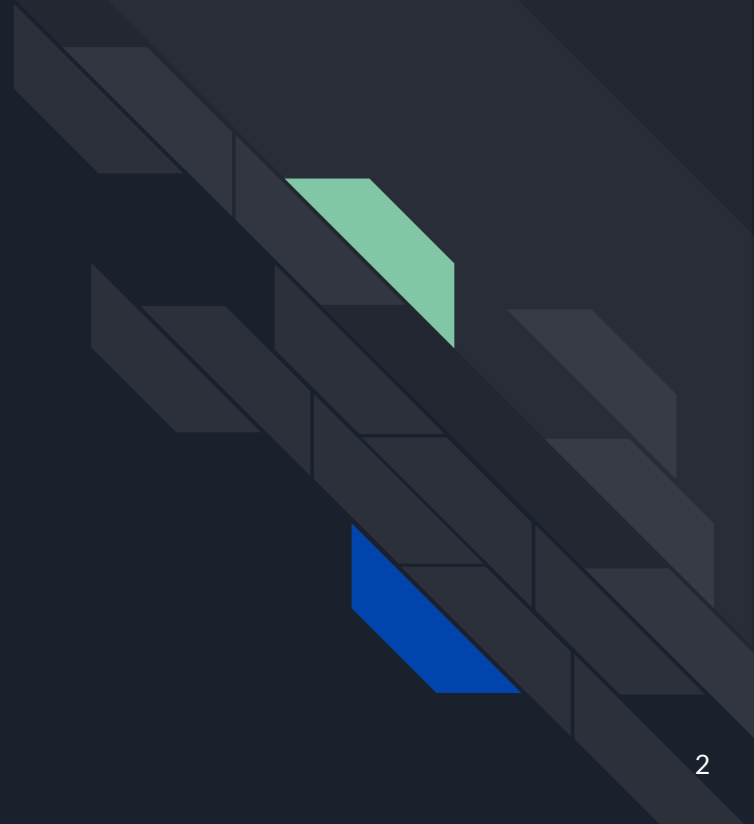




Développer pour le Cloud

2025-26 Valentin PICARD
DEVFLSTK MAST2 B

Helm avancé





HELM avancé

1. Templating conditionnel
2. Boucles
3. Fichiers embarqués
4. Service monitor
5. Alertes Prometheus
6. Alert manager
7. HPA



1 - Templating conditionnel (if/else)

Vous déployer un app sur deux environnements

- En staging, vous voulez un déploiement léger, pas de debug toolbar, pas de Sentry, logs verbeux etc ...
- En production, vous voulez Sentry actif et une variable d'environnement `ENABLE_RATE_LIMITING=true` qui n'existe pas en staging.

Avec du YAML brut, vous maintenez deux fichiers quasi-identiques.

Avec Helm, vous écrivez un seul template et vous conditionnez ce qui change.



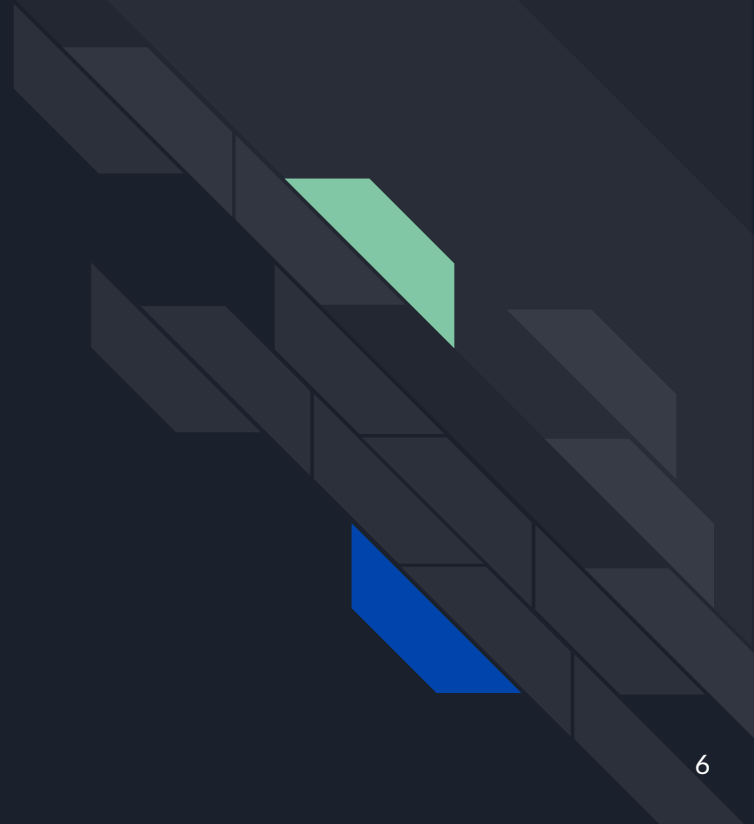
1 - Templating conditionnel (if/else)

```
{{- if .Values.sentry.enabled }}  
- name: SENTRY_DSN  
  value: {{ .Values.sentry.dsn }}  
{{- end }}
```

Le `{{- (tiret collé)` supprime les espaces et sauts de ligne avant le bloc `—` indispensable pour produire un YAML propre en sortie.

Sans lui, Helm laisse des lignes vides qui peuvent casser le rendu.

Exemple complet





```
# helm/shop/templates/payment-service.yaml
# [...]
spec:
  containers:
    - name: payment-service
      image: "shop/payment-service:{{ .Values.image.tag }}"
      env:
        - name: NODE_ENV
          value: {{ .Values.environment }}
        {{- if .Values.sentry.enabled }}
        - name: SENTRY_DSN
          value: {{ .Values.sentry.dsn }}
        {{- end }}
        - name: ENABLE_RATE_LIMITING
          value: {{ .Values.rateLimiting.enabled | quote }}
        {{- if .Values.rateLimiting.enabled }}
        - name: RATE_LIMIT_RPS
          value: {{ .Values.rateLimiting.rps | quote }}
        {{- end }}
```



```
# values.staging.yaml
environment: staging
image:
  tag: latest
paymentService:
  replicaCount: 1
sentry:
  enabled: false
rateLimiting:
  enabled: false
```

```
# values.production.yaml
environment: production
image:
  tag: v2.4.1
paymentService:
  replicaCount: 3
sentry:
  enabled: true
  dsn: "https://abc123@sentry.io/456"
rateLimiting:
  enabled: true
  rps: "100"
```



1 - Templating conditionnel (if/else)

Vérifier avant d'appliquer

```
helm template shop ./helm/shop \  
  --values ./helm/shop/values.staging.yaml \  
  --show-only templates/payment-service.yaml
```

Les blocs `{{- if false }}`...`{{- end }}` disparaissent complètement du YAML généré

- aucune ligne vide
- aucune clé vide.

Aurait-on pu
simplement écrire ça ...?



```
# helm/shop/templates/payment-service.yaml
# [...]
spec:
  containers:
    - name: payment-service
      image: "shop/payment-service:{{ .Values.image.tag }}"
      env:
        #[...]
        {{- if .Values.rateLimiting.enabled }}
        - name: ENABLE_RATE_LIMITING
          value: "true"
        - name: RATE_LIMIT_RPS
          value: {{ .Values.rateLimiting.rps | quote }}
        {{- end }}
```



```
# helm/shop/templates/payment-service.yaml
# [...]
spec:
  containers:
    - name: payment-service
      image: "shop/payment-service:{{ .Values.image.tag }}"
      env:
        #[...]
        {{- if .Values.rateLimiting.enabled }}
        - name: ENABLE_RATE_LIMITING
          value: "true"
        - name: RATE_LIMIT_RPS
          value: {{ .Values.rateLimiting.rps | quote }}
        {{- end }}
```



2 - Boucles (range)

Mettons que cette app est un app e-commerce qui expose plusieurs services.

Chacun a besoin d'un Ingress pour être accessible depuis l'extérieur sous un chemin différent :

- `/catalog` → `catalog-service`
- `/cart` → `cart-service`
- `/payment` → `payment-service`
- `/users` → `user-service`

Avec du YAML brut, vous écrivez quatre blocs de règles Ingress quasi-identiques.

Avec Helm, vous en écrivez qu'un seul.

Exemple complet



```
# values.yaml
```

```
ingress:
```

```
  host: shop.example.com
```

```
  routes:
```

- path: /catalog
 service: catalog-service
 port: 3001
- path: /cart
 service: cart-service
 port: 3002
- path: /payment
 service: payment-service
 port: 3003
- path: /users
 service: user-service
 port: 3004



```
# helm/shop/templates/ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop-ingress
  namespace: {{ .Release.Namespace }}
spec:
  rules:
    - host: {{ .Values.ingress.host }}
      http:
        paths:
          {{- range .Values.ingress.routes }}
          - path: {{ .path }}
            pathType: Prefix
            backend:
              service:
                name: {{ .service }}
                port:
                  number: {{ .port }}
          {{- end }}
```



2- Boucles (range)

Comment accéder aux values globales depuis un `range` ?

Dans un `range`, `.` est réaffecté. Pour accéder aux `Values` globales, utilisez `$`

```
{{- range .Values.ingress.routes }}  
- path: {{ .path }}  
  # ← ici on est dans le contexte de la route courante  
  # pour accéder à quelque chose hors de la liste :  
  # $.Values.ingress.host ($ = contexte racine)  
  # .service                (. = élément courant)  
{{- end }}
```



2- Boucles (range)

Vérifiez le résultat avec `helm template`

```
helm template shop ./helm/shop \  
  --values ./helm/shop/values.yaml \  
  --show-only templates/ingress.yaml
```

- Helm génère un seul objet Ingress avec quatre règles.
- Ajouter un service = ajouter une entrée dans les values.



2- Boucles (range)

Vérifiez le résultat avec `helm template`

```
helm template shop ./helm/shop \  
  --values ./helm/shop/values.yaml \  
  --show-only templates/ingress.yaml
```

- Helm génère un seul objet Ingress avec quatre règles.
- Ajouter un service = ajouter une entrée dans les values.



3 - Fichiers embarqués (.Files)

Mettons qu'un de vos services ait besoin d'un fichier `nginx.conf` de 80 lignes.

Vous devez le livrer dans un ConfigMap pour qu'il soit monté dans le Pod.

Option naïve : coller le contenu dans le template avec `| <bloc-YAML>`.

Problèmes :

- Le fichier de config est noyé dans du YAML (difficile à éditer, pas de coloration syntaxique)
- L'indentation YAML doit être parfaite sur 80 lignes
- Si vous avez 5 fichiers de config, c'est 400 lignes dans un template



```
# templates/payment-nginx-configmap.yaml
```

```
data:
```

```
  nginx.conf: |
```

```
    worker_processes auto;
```

```
    events {
```

```
      worker_connections 1024;
```

```
    }
```

```
    http {
```

```
      # ... 75 lignes de config ...
```

```
    }
```



3 - Fichiers embarqués (.Files)

Helm permet d'embarquer des fichiers présents dans le dossier du chart directement dans un template, via l'objet `.Files`.

```
helm/shop/
├── Chart.yaml
├── values.yaml
├── templates/
│   └── payment-nginx-configmap.yaml
└── config/
    ├── nginx.conf           ← votre fichier éditable
    ├── nginx-ssl.conf
    └── cors-policy.conf
```



```
# templates/payment-nginx-configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: payment-nginx-config
  namespace: {{ .Release.Namespace }}
data:
  nginx.conf: |
{{ .Files.Get "config/nginx.conf" | indent 4 }}
```

.Files.Get "config/nginx.conf" lit le contenu du fichier tel quel
indent 4 l'indente pour respecter la structure YAML du ConfigMap



```
# templates/payment-nginx-configmap.yaml
```

```
apiVersion: v1
```

```
kind: ConfigMap
```

```
metadata:
```

```
  name: payment-nginx-config
```

```
  namespace: {{ .Release.Namespace }}
```

```
data:
```

```
  {{- range $path, $content := .Files.Glob "config/*.conf" }}
```

```
  {{ base $path }}: |
```

```
    {{ $content | toString | indent 8 }}
```

```
  {{- end }}
```



3 - Fichiers embarqués (.Files)

Pour embarquer tous les fichiers d'un dossier sans modifier le template à chaque ajout :

data:

```
{{- range $path, $content := .Files.Glob "config/*.conf" }}  
  {{ base $path }}: |  
    {{ $content | toString | indent 8 }}  
{{- end }}
```

- `.Files.Glob "config/*.conf"` retourne un dictionnaire `{chemin: contenu}` pour chaque fichier correspondant au pattern
- `base $path` extrait juste le nom du fichier (`nginx.conf`), sans le chemin
- `toString` convertit le contenu binaire en string



4- ServiceMonitor

Dans une stack Docker Compose, on configure Prometheus avec un fichier `prometheus.yml` statique :

```
scrape_configs:
  - job_name: 'payment-service'
    static_configs:
      - targets: ['payment-service:3003']
```

- Dans Kubernetes, les Pods naissent et meurent dynamiquement.
- Un Pod redémarré prend une nouvelle IP. Un déploiement crée temporairement des Pods supplémentaires.
- Une config statique est aveugle à tout ça.



4- ServiceMonitor

Quand vous installez `kube-prometheus-stack`, il déploie un **Prometheus Operator**.

- Le **Prometheus Operator** étend l'API Kubernetes avec des **Custom Resource Definitions (CRDs)**,

Dont le **ServiceMonitor**

- Un **ServiceMonitor** est une ressource Kubernetes (comme un **Deployment** ou un **Service**)
- Il dit à Prometheus : "scrape tous les Services qui ont ce label, sur ce port, à cet intervalle".



4 - ServiceMonitor

Pour permettre au `ServiceMonitor` de cibler un Service, deux conditions doivent être remplies:

Un ServiceMonitor dit à Prometheus : "scrape tous les Services qui ont ce label, sur ce port ..."

- Le Service doit avoir un **label** dans ses métadonnées (pour que le selector du ServiceMonitor le trouve)
- Le port doit avoir un **nom** (pas juste un numéro)



```
apiVersion: v1
kind: Service
metadata:
  name: payment-service
  namespace: production
  labels:
    app: payment-service
spec:
  ports:
    - name: http
      port: 3003
      targetPort: 3003
  selector:
    app: payment-service
```

← condition 1 : label présent

← condition 2 : port nommé



```
apiVersion: monitoring.coreos.com/v1
```

```
kind: ServiceMonitor
```

```
metadata:
```

```
  name: payment-service-monitor
```

```
  namespace: monitoring
```

```
  labels:
```

```
    release: monitoring      # ← Prometheus Operator cherche ce label
```

```
spec:
```

```
  selector:
```

```
    matchLabels:
```

```
      app: payment-service  # ← cible les Services avec ce label
```

```
  endpoints:
```

```
    - port: http            # ← nom du port déclaré sur le Service
```

```
      path: /metrics
```

```
      interval: 15s
```

```
  namespaceSelector:
```

```
    matchNames:
```

```
      - production          # ← dans quel namespace chercher les Services
```



4 - ServiceMonitor

Par défaut, Prometheus ne surveille que son propre namespace (`monitoring`).

L'application tourne sur un namespace différent

Il faut que Prometheus découvre les `ServiceMonitor` dans d'autres namespaces



4 - ServiceMonitor

Flux complet :

```
ServiceMonitor (namespace: monitoring)
  ↓ selector: app=payment-service, namespace: production
Service payment-service (namespace: production)
  ↓ port http → 3003
Pod payment-service → GET /metrics
  ↓
Prometheus collecte → métriques stockées
```



```
# values.monitoring.yaml
```

```
kube-prometheus-stack:
```

```
  prometheus:
```

```
    prometheusSpec:
```

```
      serviceMonitorNamespaceSelector: {}      # {} = tous les namespaces
```

```
      serviceMonitorSelector:
```

```
        matchLabels:
```

```
          release: monitoring      # filtre sur le label du ServiceMonitor
```

Un nouveau développeur rejoint
l'équipe et déploie un tout
nouveau service
`recommandation-service` dans
le namespace `production`.

Que faut-il faire pour que
Prometheus scrape
automatiquement ce service ?

Un nouveau développeur rejoint
l'équipe et déploie un tout
nouveau service
`recommandation-service` dans
le namespace `production`.

Que faut-il faire pour que
Prometheus scrape
automatiquement ce service ?



5 - Alertes Prometheus

Prometheus évalue en continu des expressions PromQL.

Quand une expression est vraie pendant une durée définie (`for`), l'alerte passe en état `firing` et est transmise à Alertmanager.

Avec le Prometheus Operator, les règles sont des ressources Kubernetes natives : `PrometheusRules`



```
apiVersion: monitoring.coreos.com/v1
```

```
kind: PrometheusRule
```

```
metadata:
```

```
  name: shop-alerts
```

```
  namespace: monitoring
```

```
  labels:
```

```
    release: monitoring          # ← Prometheus Operator cherche ce label
```

```
spec:
```

```
  groups:
```

```
    - name: shop.rules
```

```
      rules:
```

```
        - alert: PaymentServiceDown
```

```
          expr: absent(up{job="payment-service"}) or up{job="payment-service"} == 0
```

```
          for: 1m
```

```
          labels:
```

```
            severity: critical
```

```
            team: backend
```

```
          annotations:
```

```
            summary: "payment-service est indisponible"
```



5 - Alertes Prometheus

Champ	Rôle
<code>alert</code>	Nom de l'alerte — apparaît dans Alertmanager et les notifications
<code>expr</code>	Expression PromQL — vraie = alerte candidate
<code>for</code>	Durée pendant laquelle l'expression doit rester vraie avant déclenchement
<code>labels</code>	Méta-données utilisées par Alertmanager pour le routing (<code>severity</code> , <code>team...</code>)
<code>annotations</code>	Messages lisibles pour les humains (<code>summary</code> , <code>description</code>)



5 - Alertes Prometheus

États d'une alerte

- Expression fausse → inactive
- Expression vraie → pending (le compteur "for" s'incrémente)
- "for" écoulé → firing → envoyée à Alertmanager
- Expression redevient fausse → resolved → Alertmanager peut notifier la résolution.



6 - Alertmanager

Prometheus déclare les alertes. Il ne les envoie pas lui-même.

Il délègue à **Alertmanager** le soin de :

- **Dédupliquer** — ne pas envoyer 50 fois la même alerte si Prometheus la réévalue toutes les 15s
- **Grouper** — envoyer un seul email si plusieurs services tombent en même temps
- **Silencer** — inhiber les notifications pendant une maintenance planifiée
- **Router** — envoyer au bon destinataire selon les labels (`severity`, `team...`)



```
# helm/monitoring/templates/alertmanager-config.yaml
```

```
apiVersion: v1
```

```
kind: Secret
```

```
metadata:
```

```
  name: shop-alertmanager-config
```

```
  namespace: monitoring
```

```
stringData:
```

```
  alertmanager.yaml: |
```

```
    [... Config : next slide ... ]
```



```
global:
  smtp_smarthost: 'smtp-relay.example.com:587'
  smtp_from: '{{ .Values.alertmanager.smtp.from }}'
  smtp_auth_username: '{{ .Values.alertmanager.smtp.username }}'
  smtp_auth_password: '{{ .Values.alertmanager.smtp.password }}'

route:
  group_by: ['alertname', 'team']
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  receiver: 'email-ops'
  routes:
    - match:
        severity: critical
        team: backend
      receiver: 'pagerduty-backend'

receivers:
  - name: 'email-ops'
    email_configs:
      - to: '{{ .Values.alertmanager.smtp.to }}'
        send_resolved: true
  - name: 'pagerduty-backend'
    pagerduty_configs:
      - routing_key: '{{ .Values.alertmanager.pagerduty.key }}'
```



6 - Alert manager

Alerte "PaymentServiceDown"

labels: severity=critical, team=backend

↓

Route principale → cherche une route spécifique

↓ match: severity=critical AND team=backend → oui

Route "pagerduty-backend"

↓ group_wait: 30s (attend d'autres alertes à grouper)

PagerDuty notifié

Alerte "HighMemoryUsage"

labels: severity=warning, team=infra

↓

Route principale → cherche une route spécifique

↓ aucune route ne match

Receiver par défaut "email-ops"

↓

Email envoyé

La base de données est down,
vous recevez 12 alertes en même
temps.

Tous les services de votre app ne
répondent plus.

Sans le mécanisme de
groupement d'Alertmanager,
que se passe-t-il ?

La base de données est down,
vous recevez 12 alertes en même
temps.

Tous les services de votre app ne
répondent plus.

Sans le mécanisme de
groupement d'Alertmanager,
que se passe-t-il ?



7 - HPA (Horizontal Pod Autoscaler)

- Sous un pic de trafic, un ou plusieurs services saturent.
- Le alert manager peut notifier l'équipe DevOps et la réaction naturelle : augmenter le nombre de réplicas.
- `kubectl scale deployment catalog-service -n production --replicas=5`

Deux problèmes fondamentaux :

- **Réactif**, quelqu'un doit surveiller et agir, souvent trop tard
- **Irréversible**, si le trafic baisse, les 5 replicas tournent jusqu'au prochain scale down manuel

Le Horizontal Pod Autoscaler résout les deux.



7 - HPA (Horizontal Pod Autoscaler)

Le HPA est un contrôleur Kubernetes qui observe une métrique sur un Deployment

Il ajuste `spec.replicas` automatiquement pour rester dans une plage définie.

On peut définir une règle d'observation sur le CPU par exemple :

- HPA observe → CPU moyen des Pods du service
- Si CPU > 70% → scale up (nouveaux Pods créés)
- Si CPU < 70% → scale down (Pods excédentaires supprimés)
- Toujours entre les deux variables : `minReplicas` et `maxReplicas`



7 - HPA (Horizontal Pod Autoscaler)

Sur l'exemple du CPU, le HPA a besoin de lire les métriques des Pods en temps réel.

Il les obtient via le **Metrics Server**

C'est un composant qui agrège les données exposées par `kubelet` sur chaque nœud.

Pour vérifier que le Metrics Server est actif :

```
kubectl top pods -n <namespace>
```



```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: catalog-service-hpa
  namespace: production
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: catalog-service
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70      # % de la cpu request du container
```



7 - HPA (Horizontal Pod Autoscaler)

Par conception, le HPA est **rapide à monter, lent à descendre**

- **Scale up** : réagit après 3 minutes de charge élevée (évite de scaler sur un spike de 30 secondes)
- **Scale down** : attend 5 minutes de charge basse avant de supprimer des Pods (évite le yo-yo)

Ces délais sont configurables via `behavior`



```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: catalog-service-hpa
  namespace: production
spec:
  # [...]
  metrics:
    # [...]
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
      # 5 min de stabilisation avant de descendre
    policies:
      - type: Pods
        value: 1
        periodSeconds: 60
        # max 1 Pod supprimé par minute
    scaleUp:
      stabilizationWindowSeconds: 180
      # 3 min avant de monter
```

À 4h du matin, le trafic est quasi nul.

Avec cette configuration du HPA.

Que fait le HPA ?

À 4h du matin, le trafic est quasi nul.

Avec cette configuration du HPA.

Que fait le HPA ?



7 - HPA

Le CPU est facile à obtenir (Metrics Server, pas de config supplémentaire), mais c'est un signal **indirect**.

Exemple :

- Un service fait des appels vers une API externe de tarification.
- Sous charge, le CPU reste à 20% (le service passe son temps à attendre la réponse HTTP)
- La latence monte à 3 secondes et les utilisateurs abandonnent le panier.
- Le HPA sur CPU ne voit pas de problème, aucune adaptation n'est faite.

La métrique qui compte, c'est celle qui représente directement la charge réelle :

- **Le nombre de requêtes par seconde traitées par chaque Pod.**



7 - HPA

La latence p95 est encore plus représentative de l'expérience utilisateur.

Mais elle pose un problème fondamental :

- **elle n'est pas linéairement liée au nombre de Pods.**

Si la latence est haute parce que l'API externe de tarification est lente, doubler le nombre de Pods du service qui appelle cet API ne change rien

Chaque Pod attend 800ms la réponse de l'API tierce. En avoir 2 ou 10 ne rendrait pas cette API plus rapide

Le bottleneck n'est pas dans notre service.



7 - HPA

La latence p95 est souvent plus utile comme **signal d'alerte Alertmanager**

On notifie l'équipe pour qu'elle investigue

Comme signal de scaling, l'action automatique :

- Est inefficace
- Peut aggraver les choses en surchargeant la dépendance externe avec plus de requêtes parallèles



7 - HPA

Métrique	Avantages	Limites
CPU	Simple, toujours disponible	Indirect — invisible si le service attend une I/O
Requêtes/s	Direct, réactif à la charge réelle	Nécessite Prometheus Adapter
Latence p95	Le plus proche de l'expérience utilisateur	Scaling potentiellement inefficace si bottleneck externe



Ressources utiles

Prometheus Operator & monitoring

- [ServiceMonitor — doc officielle](#) — le CRD vu en cours
- [PrometheusRule — doc officielle](#) — définir ses alertes comme ressource Kubernetes

HPA

- [HPA — doc Kubernetes](#) — référence autoscaling/v2, behavior, métriques
- [Prometheus Adapter](#) — exposer des métriques custom PromQL au HPA



6 - Alertmanager

Configuration : le problème du Secret

- Le Prometheus Operator (de la dépendance Helm : `kube-prometheus-stack`) régénère son Secret de configuration.

La solution :

- créer un Secret, et référencer son nom via `configSecret`.



```
# helm/monitoring/templates/alertmanager-config.yaml
```

```
apiVersion: v1
```

```
kind: Secret
```

```
metadata:
```

```
  name: shop-alertmanager-config
```

```
  namespace: monitoring
```

```
stringData:
```

```
  alertmanager.yaml: |
```

```
    [... Config : next slide ... ]
```

```
# values.monitoring.yaml
```

```
kube-prometheus-stack:
```

```
  alertmanager:
```

```
    alertmanagerSpec:
```

```
      # nom du Secret créé par notre template
```

```
      configSecret: shop-alertmanager-config
```



8 - HPA

On veut utiliser une metrics Prometheus custom

Le HPA ne sait pas parler à Prometheus directement.

Il sait interroger l'API Kubernetes `custom.metrics.k8s.io`

Un composant intermédiaire, **Prometheus Adapter**, fait le pont :

- Il sait lire les métriques Prometheus
- Il sait exposer ces métriques sur l'API `custom.metrics.k8s.io`



8 - HPA

Lire une metrique Prometheus Custom implique :

- L'installation du Prometheus Adapter



8 - HPA

Lire une metrique Prometheus Custom implique :

- L'installation du Prometheus Adapter
- Enseigner au Prometheus Adapter comment transformer une requête PromQL en métrique lisible par le HPA



8 - HPA

Lire une metrique Prometheus Custom implique :

- L'installation du Prometheus Adapter
- Enseigner au Prometheus Adapter comment transformer une requête PromQL en métrique lisible par le HPA
- Vérifier que la métrique est exposée



8 - HPA

Lire une metrique Prometheus Custom implique :

- L'installation du Prometheus Adapter
- Enseigner au Prometheus Adapter comment transformer une requête PromQL en métrique lisible par le HPA
- Vérifier que la métrique est exposée
- Écrire le HPA avec sa métrique custom

On configure le HPA pour que chaque Pod traite maximum 150 req/s.

Le trafic monte à 900 req/s.
Combien de Pods le HPA va-t-il créer ?